The Journal of Open Source Software

Foundations for nighttime lights data analysis

Ayush Patnaik[1*], **Ajay Shah**[1*], **and Susan Thomas**[1*]

1 xKDR Forum, India * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Mehmet Hakan Satman

Submitted: 12 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

A remarkable development in alternative data is the use of satellites that measure nighttime light radiance. From the early 1990s, Nightlights (NL) has been used to measure economic activity. NL correlates well with standard measures of growth and development (Doll et al., 2006), which allows measures of economic growth where existing measures are either unreliable or lack the continuity and/or resolution of constant satellite observation.

Raw monthly NL files are not analysis-ready. They contain missing observations, negative values, outliers, background noise, and cloud-related attenuation (Jiang et al., 2017; Ma et al., 2014; Uprety et al., 2019). As a result, every research team must individually implement preprocessing, often with project-specific scripts. This creates entry barriers and weakens comparability and reproducibility across studies.

Statement of need

`NighttimeLights.jl` is a Julia package for cleaning and bias-correcting monthly nighttime lights from the Visible Infrared Imaging Radiometer Suite (VIIRS). It provides a transparent and reproducible workflow that implements established methods from the applied literature, including cloud-related attenuation correction (Gupta et al., 2017; Patnaik et al., 2021; Pearson, 2002).

The package addresses a practical gap: many papers using NL data rely on individual preprocessing pipelines, making it difficult to reproduce findings or compare outcomes across studies. By consolidating key cleaning steps into one standard package, `NighttimeLights.jl` reduces duplicated effort and standardizes core data preparation.

State of the field

There are two dominant nighttime-lights products used in applied work: annual DMSP-OLS composites (1992-2013) and monthly VIIRS composites (2012-present). VIIRS has finer temporal and spatial resolution and is now the primary data source for contemporary analysis.

Most empirical NL workflows use combinations of missing-value recoding, outlier removal, and cloud-aware filtering, but implementation details vary across projects. This heterogeneity limits comparability. `NighttimeLights.jl` standardizes these operations while retaining modular control for users.

Several software tools already serve the nighttime-lights community, but they address a different part of the workflow. `blackmarbler` (Marty & Stefanini Vicente, 2024) and its Python counterpart `BlackMarblePy` (Stefanini Vicente & Marty, 2024) download NASA's Black Marble product and compute zonal statistics for administrative regions; `Rnightlights` (Njuguna, 2019) does the same for the NOAA DMSP-OLS and VIIRS composites; and the VIIRS collections in Google Earth Engine (Gorelick et al., 2017) provide server-side access

DRAFT

and aggregation. These tools focus on acquisition and spatial aggregation, and the Black Marble product applies sensor, terrain, and atmospheric corrections upstream at the pixel level. None of them implement a configurable statistical cleaning pipeline for the raw NOAA monthly composites, and none correct the cloud-cover attenuation bias that dominates monthly series in monsoonal regions. Contributing these routines to the existing packages would mean maintaining the same methods across three language ecosystems; providing them as a standalone Julia package, with the cleaning pipeline itself as the unit of reuse, keeps the methods in one place and keeps a cleaned panel reproducible from a single package version.

Software design

`NighttimeLights.jl` is implemented in Julia and built on `Rasters.jl` for geospatial data handling. Data is represented as raster data cubes with dimensions (x, y, t), and cleaning is organized as composable functions. Core functions include:

- Missing-data handling: `na_recode`, `na_interp_linear`
- Negative-value handling: `replace_negative`
- Outlier treatment: `outlier_hampel`, `outlier_variance`
- Background-noise filtering: `bgnoise_PSTT2021`
- Cloud-bias correction: `bias_PSTT2021`

These are wrapped into high-level pipelines such as `PSTT2021()`, `PSTT2021_conventional()`, and `clean_complete()`. This design supports both method-level customization and standardized cleaning pipelines.

Several design choices shaped the package. Primarily, cleaning is exposed as a set of small composable functions rather than a single opaque routine. This enlarges the API surface and requires the user to respect an ordering between steps, but it allows for customization and lets each stage be run, inspected, and switched out independently, which is what makes the package usable for methodological work while still allowing reproducability. Secondly, the package is written in Julia because the core operations are per-pixel time-series computations over large pixel counts. Hampel filters, smoothing-spline fits, linear interpolation are awkward to express as fast vectorised array code and would otherwise require C extensions, if not for Julia's loop performance. Finally, existing best-practices cleaning and cloud-bias-corrected cleaning are kept as separately named and dated pipelines (`PSTT2021_conventional` and `PSTT2021`) so that results remain reproducable as well as directly comparable to the large body of prior work that does not correct for cloud cover.

Research impact statement

The package has been used as infrastructure for empirical work where cloud cover and data quality strongly affect inference from nighttime lights. Its main impact is methodological consistency: researchers can move from raw files to analysis-ready panels using explicit, shared preprocessing steps.

Because each stage is exposed as a public function, `NighttimeLights.jl` also serves as a platform for methodological development. Users can inspect intermediate outputs, compare alternative cleaning choices, and contribute new preprocessing methods while preserving reproducible workflows.

The cloud-bias correction implemented here was developed and validated in Patnaik et al. (2021), and `NighttimeLights.jl` is the reference implementation of that method. The package has been used to build analysis-ready VIIRS panels for applied economic work, including a study of wartime economic activity in Ukraine (Hande et al., 2025) and an analysis of regional economic patterns across Java, Indonesia (Aviliani et al., 2026). It also underpins the public India Built and Lit dataset, which distributes cleaned nighttime-lights panels for India (xKDR

DRAFT

Forum, 2026). Reproducibility is supported concretely: a Mumbai example cube ships with the package and is loaded with `load_example()`, every pipeline stage is a documented public function, the package is registered in the Julia General registry, and the results quoted in this paper were produced with the pinned versions listed under Computational details.

Computational details

Results referenced in this manuscript were generated with Julia 1.12.6 and `NighttimeLights.jl` 0.9.1. Version 0.10.0, described here, changes fewer than 0.1% of the cleaned values in the bundled Mumbai example. Julia is available at https://julialang.org/ and the package is available at https://github.com/xKDR/NighttimeLights.jl.

Installation from the General registry:

```
] add NighttimeLights
```

Usage example

The following Julia code shows how to go from raw to cleaned data with `NighttimeLights.jl`. The example uses monthly VIIRS composites for a rectangle around Mumbai, January 2014 to January 2018, downloaded from Google Earth Engine by the replication notebook at https://github.com/xKDR/NighttimeLights.jl/blob/main/submission/SRC/replication.ipynb. Two inputs are needed. Both are `Raster` data cubes with dimensions (x, y, t), where t indexes months:

- `mumbai_radiance`: the monthly mean radiance of each pixel, in $nW\ cm^{-2}\ sr^{-1}$.
- `mumbai_n_cloudfree`: the number of cloud-free observations that went into each monthly mean.

The simplest path is a single call:

```
mumbai_cleaned = clean_complete(mumbai_radiance, mumbai_n_cloudfree, bgthreshold=4)
```

which is presently identical to:

```
mumbai_cleaned = PSTT2021(mumbai_radiance, mumbai_n_cloudfree, bgthreshold=4)
```

Both wrap the sequence of steps below. Each step is a public function, so the sequence can be run one step at a time, inspected, or modified:

```
# 1. Missing data: months with no cloud-free observations become `missing`
radiance_na = na_recode(mumbai_radiance, mumbai_n_cloudfree)

# 2. Negative values: recoded as `missing`
radiance_nonneg = replace_negative(radiance_na)

# 3. Background noise: mask out pixels that are dark in the annual image
lit_mask = bgnoise_PSTT2021(radiance_nonneg, mumbai_n_cloudfree, 4)
radiance_lit = apply_mask(radiance_nonneg, lit_mask)

# 4. Cross-sectional outliers: mask out pixels with extreme variance over time
stable_mask = outlier_variance(radiance_lit, lit_mask)
radiance_stable = apply_mask(radiance_lit, stable_mask)

# 5. Time-series outliers: Hampel filter on each pixel's time series
radiance_despiked = long_apply(outlier_hampel, radiance_stable)

# 6. Cloud bias: correct attenuation in months with few cloud-free observations
```

DRAFT

```
pixel_mask = lit_mask .* stable_mask
radiance_corrected = bias_PSTT2021(radiance_despiked, mumbai_n_cloudfree, pixel_mask)

# 7. Gaps: linearly interpolate the remaining `missing` values
mumbai_cleaned = long_apply(na_interp_linear, radiance_corrected)
```

Two helpers recur: `apply_mask` removes the pixels a 2D mask marks as `missing` from every month, and `long_apply` applies a function written for one pixel's time series to every pixel. The steps are:

1. **Missing data.** NOAA reports zero radiance for a pixel-month with no cloud-free observations, which is indistinguishable from a dark pixel. `na_recode` recodes these as `missing`.
2. **Negative values.** These arise from calibration in the presence of airglow (Uprety et al., 2019) and are recoded as `missing`.
3. **Background noise.** Unlit areas such as oceans, forests and deserts still show a small positive radiance. `bgnoise_PSTT2021` averages the last twelve months, weighting each by its cloud-free observations (Gupta et al., 2017), and marks pixels below the threshold (here 4 $nW\ cm^{-2}\ sr^{-1}$) as dark.
4. **Cross-sectional outliers.** Fires and gas flares produce pixels with extreme, highly variable radiance. `outlier_variance` marks the top 0.1% of lit pixels by the standard deviation of their detrended series as unstable.
5. **Time-series outliers.** `outlier_hampel` removes isolated spikes with a Hampel filter (Pearson, 2002), using a 5-month window and a cutoff of 3 standard deviations.
6. **Cloud bias.** Cloudy months show attenuated radiance: the monsoon dip in the figure below. For each pixel kept by steps 3 and 4, `bias_PSTT2021` fits a smoothing spline of detrended radiance on the number of cloud-free observations, and raises cloudy months to the level the spline predicts for clear months (Patnaik et al., 2021).
7. **Remaining gaps.** `na_interp_linear` fills gaps by linear interpolation; leading and trailing gaps take the nearest observed value.

Steps 1 to 5 and 7, without the bias correction, are what most applied work already does. They are termed conventional cleaning in Patnaik et al. (2021) and are available as `PSTT2021_conventional()`.

DRAFT

An example

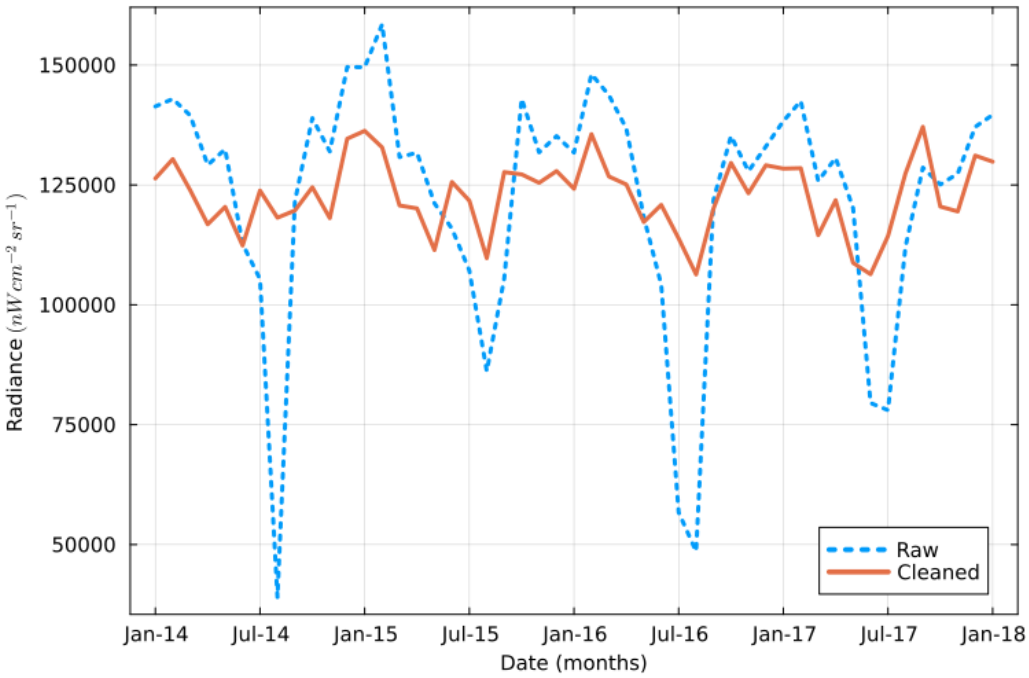


Figure 1: Aggregate radiance of raw and cleaned nighttime lights for Mumbai, January 2014 to January 2018.

Figure 1 shows the aggregate raw and cleaned radiance of Mumbai, in $nW\,cm^{-2}\,sr^{-1}$, from January 2014 to January 2018; the notebook that produced it is at https://github.com/xKDR/NighttimeLights.jl/blob/main/submission/SRC/replication.ipynb. The monsoon dip (e.g. July) in the raw data is much reduced after cleaning.

Table 1: Effect of cleaning on nighttime lights for Mumbai.

Statistic	Raw	Cleaned
Minimum	-0.14	0.0
Mean	9.1	9.1
Median	3.2	0.0
99th percentile	54.0	59.0
Maximum	4300.0	1700.0

After cleaning there are no negative values, and the maximum is lower because outliers are removed. Half the raw values are below 4, attributable to background noise, and are zeroed. The higher 99th percentile reflects the bias correction.

AI usage disclosure

Generative AI (Anthropic's Claude, through Claude Code) assisted with parts of the test suite, documentation of the outlier filters' limitations, and the continuous integration configuration. The authors reviewed all AI-assisted changes and take responsibility for them. AI assistance was also used in formatting tables in the markdown document.

DRAFT

The Journal of Open Source Software

Acknowledgements

We acknowledge contributions from Anshul Tayal, Siddhant Chauddhary, and Hrishikesh Saikia for their contributions to the codebase and package design, and Kapilan Mahalingam for package improvements, test development, and manuscript preparation. This work received financial support from the MIT Julia Lab.

References

Aviliani, A., Kartawinata, M., Herawan, J. E., Parningotan, F., & Komala, A. (2026). When lights tell stories: The untold economic pattern of Java island. *Jurnal Ekonomi Indonesia*, *15*(1), 1–18. https://doi.org/10.52813/jei.v15i1.786

Doll, C. N., Muller, J.-P., & Morley, J. G. (2006). Mapping regional economic activity from night-time light satellite imagery. *Ecological Economics*, *57*(1), 75–92. https://doi.org/10.1016/j.ecolecon.2005.03.007

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, *202*, 18–27. https://doi.org/10.1016/j.rse.2017.06.031

Gupta, P. K., Srivastav, S. K., Sesha Sai, M. V. R., Gharai, B., Senthil Kumar, A., & Krishna Murthy, Y. V. N. (2017). Analysis of SNPP-VIIRS-DNB derived nightlights over India. *38th Asian Conference on Remote Sensing: Space Applications — Touching Human Lives*.

Hande, R., Patnaik, A., Shah, A., & Thomas, S. (2025). *Shedding light on the Russia-Ukraine war* (No. 40). xKDR Forum. https://www.xkdr.org/paper/shedding-light-on-the-russia-ukraine-war

Jiang, W., He, G., Long, T., & Liu, H. (2017). Ongoing conflict makes yemen dark: From the perspective of nighttime light. *Remote Sensing*, *9*(8). https://doi.org/10.3390/rs9080798

Ma, T., Zhou, C., Pei, T., Haynie, S., & Fan, J. (2014). Responses of suomi-NPP VIIRS-derived nighttime lights to socioeconomic activity in china's cities. *Remote Sensing Letters*, *5*(2), 165–174. https://doi.org/10.1080/2150704x.2014.890758

Marty, R., & Stefanini Vicente, G. (2024). *blackmarbler: Black Marble data and statistics*. R package, Comprehensive R Archive Network. https://doi.org/10.32614/CRAN.package.blackmarbler

Njuguna, C. (2019). *Rnightlights: Satellite nightlight data extraction*. R package, Comprehensive R Archive Network (archived).

Patnaik, A., Shah, A., Tayal, A., & Thomas, S. (2021). *But clouds got in my way: Bias and bias correction of viirs nighttime lights data in the presence of clouds* (No. 7). xKDR Forum. https://doi.org/10.2139/ssrn.3957319

Pearson, R. K. (2002). Outliers in process modeling and identification. *IEEE Transactions on Control Systems Technology*, *10*(1), 55–63. https://doi.org/10.1109/87.974338

Stefanini Vicente, G., & Marty, R. (2024). *BlackMarblePy: Georeferenced rasters and statistics of nighttime lights from NASA Black Marble*. Python package. https://doi.org/10.5281/zenodo.10667925

Uprety, S., Cao, C., Gu, Y., Shao, X., Blonski, S., & Zhang, B. (2019). Calibration improvements in s-NPP VIIRS DNB sensor data record using version 2 reprocessing. *IEEE Transactions on Geoscience and Remote Sensing*, *57*(12), 9602–9611. https://doi.org/10.1109/tgrs.2019.2927942

xKDR Forum. (2026). *NighttimeLights and built-up volume dataset*. https://xkdr.github.io/

DRAFT

194 India-Built-and-Lit/.

DRAFT